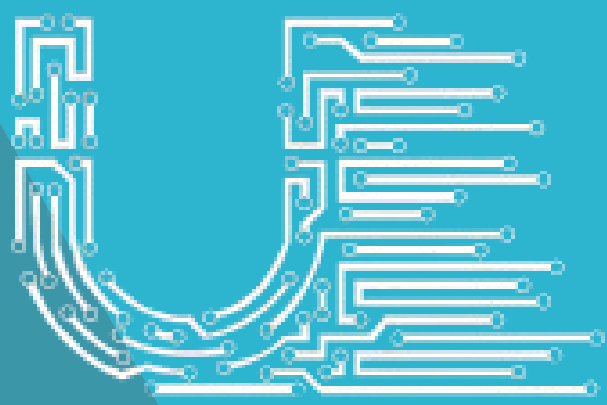




UDYAM' 26

syNNapse'26

PS1

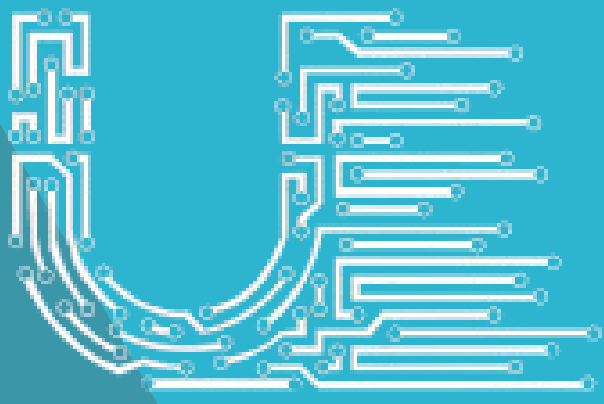
Powered by



Retina Robotics

**ELECTRONICS
ENGINEERING
SOCIETY**





UDYAM' 26

Vision-Based Inventory Item Re-Identification

Overview

The Goal

The goal is to develop a computer-powered visual recognition system that enables enterprises to identify inventory items purely based on visual similarity, without relying on labels, barcodes, or metadata.

The Context

Large enterprises such as warehouses, logistics companies, and retailers manage thousands of visually similar inventory items. Many items lack readable identifiers, or identifiers are damaged, missing, or inconsistent. Human verification does not scale.

The enterprise needs an AI system that can look at an image of an item and find the same item (or closest match) from an existing inventory database.

Core Problem

Enterprises need a scalable, accurate, and model-agnostic system to perform Inventory Item Re-Identification.

The system must perform two core functions:

1. Visual Encoding

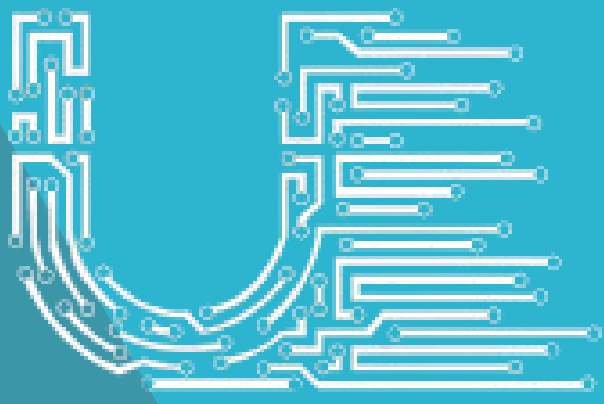
- Convert an image of an inventory item into a compact representation that captures its visual identity.

2. Similarity-Based Retrieval

- Given a query image, retrieve the most visually similar inventory item(s) from a database using similarity scoring.

**ELECTRONICS
ENGINEERING
SOCIETY**





UDYAM' 26

Here is the detailed scope of work for the project:

Module A: Visual Feature Extraction (Item Encoding)

The system must extract robust visual embeddings that uniquely represent an inventory item.

Key Requirements

a) Single-Item Assumption

- Each image contains one primary inventory item.
- No object detection or segmentation is required.

b) Backbone Flexibility

- Participants may use CNN backbones (e.g., ResNet, MobileNet).
- Transformer backbones (e.g., ViT, DeiT, Swin).

c) Visual Robustness

- Embeddings should remain stable across:
 - Viewpoint changes
 - Lighting variations
 - Background clutter
 - Minor occlusions

Module B: Similarity Scoring & Retrieval

Once embeddings are generated, the system must perform similarity-based matching.

Input

- A query image of an inventory item
- A pre-computed database of inventory item embeddings

Process

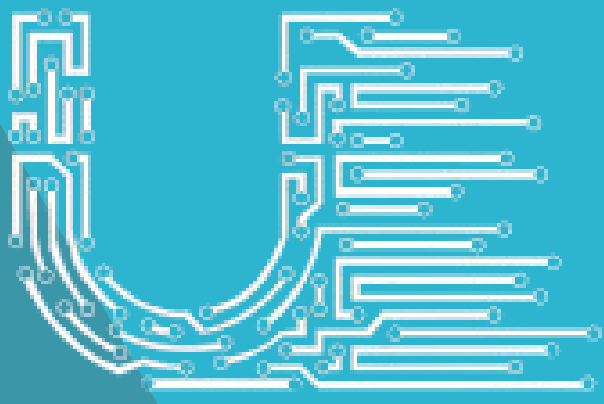
- Compare the query embedding with database embeddings
- Compute similarity scores (e.g., cosine similarity)
- Rank inventory items by similarity

Output

- Top-K most visually similar inventory items
- Associated similarity scores

**ELECTRONICS
ENGINEERING
SOCIETY**





UDYAM' 26

Glossary

Inventory Re-Identification: Matching an image of an item to the same physical item in a database.

Embedding: A fixed-length vector representing visual characteristics of an item.

Similarity Scoring: Measuring how close two embeddings are in feature space.

Open-Set Recognition: The ability to handle new items not seen during training.

Deliverables

1. GitHub Repository

Must include:

- src/feature-extraction: Folder with visual feature extraction code
- src/similarity-scoring-and-retrieval: Folder with retrieval code
- src/api (BONUS)
- requirements.txt
- docs: This folder contains all the code while you were training and testing your model in notebook format, document all the different approaches you took and how your final approach is the best among them.
- README.md

2. README.md

Must clearly explain:

- Problem approach
- Model architecture
- Similarity metric used
- Retrieval pipeline
- Why this approach was chosen

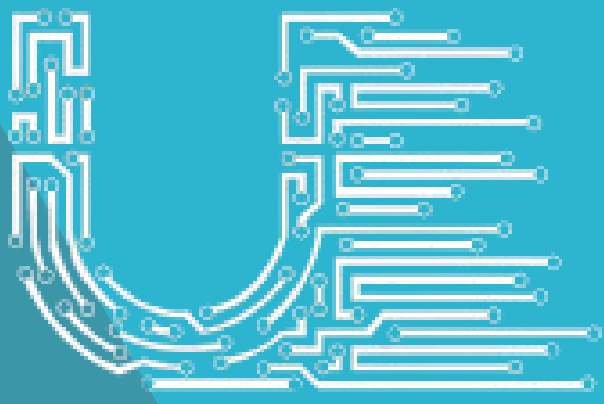
3. Sample Retrieval Output

Upload one example showing:

- Query image
- Top-K retrieved images
- Correct instance highlighted

**ELECTRONICS
ENGINEERING
SOCIETY**





UDYAM' 26

4. Evaluation Criteria (100 Points)

- 20 points for the novelty in approach.
- 50 points for the result metrics obtained.
- 30 points for code clarity and documentation.
- BONUS: If you completed the above task really well and you are confident about your model performance, then go ahead and create a Fast API of this and deploy the same on render, railway, etc. if possible or localhost is also good work but deploying is a plus. (20 points)

How to create the codebase and document the code?

While training use your jupyter notebooks normally as you do in one file do all the things but when your final embeddings are generated that is you think this is the highest you can achieve, create (.py) files for separate classes and the embedding database (ID to embedding mapping) and store them in respective folders as directed above with names being consistent with the class name. Now import the classes which are required in different files:

(from <package>.<sub_package>.<module> import <object>).

Eg: from feature_extraction.encoder import ImageEncoder

Important: Use package initializer file (`__init__.py`) in each folder inside src.

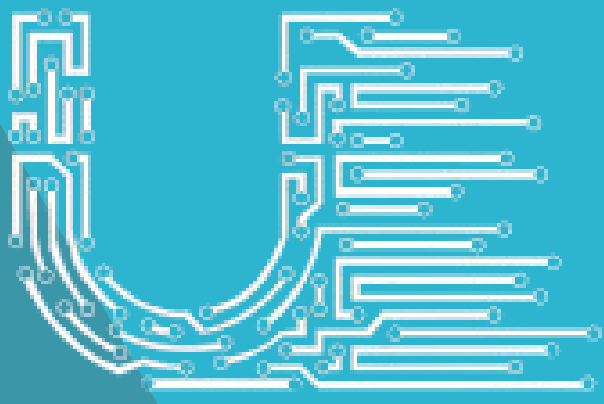
Now add the notebook, requirements.txt and README.md

BONUS: Now if you are confident with your model performance create a fastapi endpoint which could accept a request with an image and give top-k closest images as the response along with their similarity scores. All this inside src/api in one file (main.py) by importing required classes and implementing the function. Now run in bash - uvicorn

src.api.main:function_name --reload from project root to start your server on localhost (deploying on cloud (render, etc) is a plus). Use postman API or curl to send a request and receive a response, include the screenshots of them both the request you sent and the response recieved in your

README.md.

Push all this to a github repo and submit the link.



UDYAM' 26

How to Evaluate Your Model

1. Dataset Split (Instance-Level):

Begin by performing an 80–20 split based on inventory item IDs. Ensure that all images related to a specific item ID are placed entirely in either the training set or the test set, with no overlap. This approach facilitates evaluation in an open-set retrieval setting.

2. Test-Time Retrieval Setup:

The evaluation process is conducted exclusively on the test set. For each test item ID, designate one image as the query image and use the remaining test images as the retrieval gallery. The objective is for the model to accurately retrieve visually similar images of an item it has never encountered during training.

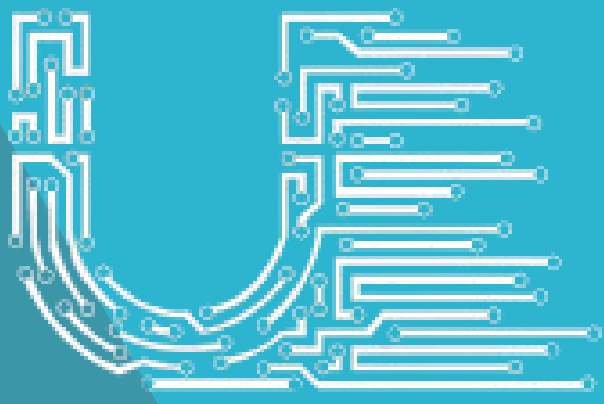
3. Evaluation Metric: Top-K Retrieval Accuracy:

For each query image, retrieve the Top-K most similar images from the test gallery. A retrieval is deemed correct if any image with the same item ID appears within the Top-K results. Performance should be reported using the following metrics:

- Top-1 Accuracy
- Top-5 Accuracy (as the primary metric)
- Top-10 Accuracy

The dataset for this task is accessible at:

<https://www.kaggle.com/datasets/liucong12601/stanford-online-products-dataset>



UDYAM' 26

Prize Pool

- 1st Place → ₹ 10,000
- 2nd Place → ₹ 7,500
- 3rd Place → ₹ 5,000
- Seperate prize pool for Freshers Goodies worth ₹ 5,000
- Final Submission Deadline : 19th Feb

Contact us for any queries:

Bhaskar Pandey : +919140427927 Dhairya Arora : +91 90273 07003

**ELECTRONICS
ENGINEERING
SOCIETY**

